Binary tree and BST — everything written down

30 unique. 30 filled in. Updated 24 Sep 2026.

Copies of the same function (printTree, insert, stack, queue) are not listed. isBSTWithHelper is the same min/max check as Is BST, so it is not its own item. findLCAKey is the same walk as Find LCA, so it is not its own item. printBSTInOrder (recursive and iterative) is the same inorder walk. insertBSTNodeVer2 is the same iterative insert. The grandchild iterative version is the same check as Has grandchild.

The walk is the part with more than one way. Each job under it uses one way. That way is written on the line.

A node has left and right. The lab field is item, data, or val. Same field. These snippets use data.

Sample

```
      1
     / \
    2   3
   / \
  4   5
```

pre	1 2 4 5 3
in	4 2 5 1 3
post	4 5 2 3 1
level	1 2 3 4 5

Inorder — left, me, right

recursive	left, then print, then right.
iterative, one stack	go left and push every node on the way. At None, pop, record, then step right.

recursive

```
if node is None:
    return
inorder(node.left)
print(node.data, end=" ")
inorder(node.right)
```

iterative, one stack

```
stack = []
current = root
while current or stack:
    while current:
        stack.append(current)
        current = current.left
    current = stack.pop()
    print(current.data, end=" ")
    current = current.right
```

trace, the sample

- push 1, push 2, push 4. pop 4, print. no right
- pop 2, print. right is 5. push 5, pop 5, print
- pop 1, print. right is 3. push 3, pop 3, print
- 4 2 5 1 3

Jobs that need inorder

These three need this order. Left has to finish before this node, or a BST has to come out sorted. They use the loop above. They are not other ways to print the tree.

Is BST by inorder

iterative. Each value must be bigger than the one before. same loop. Remember the last value. This one has to be bigger.

```
prev = None
while current or stack:
    while current:
        stack.append(current)
        current = current.left
    current = stack.pop()
    if prev is not None and current.data <= prev:
        return False
    prev = current.data
    current = current.right
return True
```

An equal key fails. The lab writes <=.

Kth smallest

iterative. Count the visits, stop at k. same loop. A BST inorder is sorted, so the kth pop is the kth smallest.

```
if k < 1:
    return -1
```

```
    while current or stack:
        while current:
            stack.append(current)
            current = current.left
        current = stack.pop()
        k -= 1
        if k == 0:
            return current.data
        current = current.right
    return -1
```

Sum-greater tree

iterative, right side first (right, me, left). Running sum. same loop with the sides swapped. Go right, push, pop, then step left. Everything already visited is bigger, so the running total is the sum of greater keys. Store that total in the node.

```
    total = 0
    current = root
    stack = []
    while current or stack:
        while current:
            stack.append(current)
            current = current.right
        current = stack.pop()
        total += current.data
        current.data = total
        current = current.left
```

trace

```
      2            5
     / \    →     / \
    1   3        6   3
```

- visit 3, total 3, node stays 3
- visit 2, total 5, node becomes 5
- visit 1, total 6, node becomes 6

The total is a local variable in that one loop. A recursive write has to return the total, because passing it as a normal argument does not update the caller.

When to use

You need the nodes in sorted order, or you need left finished before this node. Inorder on a BST is the sorted list. Stop early: count the visits. Bigger keys first: swap the sides, right then me then left.

Preorder — me, then both children

recursive	print, then left, then right.
iterative, one stack	pop, do the job now, then push the children. Push right first, then left, so left comes off next.

recursive

```
if node is None:
    return
print(node.data, end=" ")
preorder(node.left)
preorder(node.right)
```

iterative, one stack

```
if root is None:
    return
stack = [root]
while stack:
    current = stack.pop()
    print(current.data, end=" ")
    if current.right:
        stack.append(current.right)
    if current.left:
        stack.append(current.left)
```

trace, the sample

- pop 1, print. push 3, push 2
- pop 2, print. push 5, push 4
- pop 4, pop 5, pop 3
- 1 2 4 5 3

When to use

You are printing preorder: this node, then the left subtree, then the right. One stack: pop, print, push right, push left.

Print smaller than m, and has grandchild, do not need this order. The check only looks at this node. They are under "Order does not matter".

Postorder — both children, then me

You cannot record this node until both children are done.

recursive	left, right, then print.
iterative, visit twice	the main iterative one. A reminder sits on the stack. Record the node only when both sides are done.
iterative, two stacks	last. The second stack is a reverse.
iterative, one stack then reverse	last. Records too early, then flips. Reverse is not guaranteed.

recursive

```
if node is None:
    return
postorder(node.left)
postorder(node.right)
print(node.data, end=" ")
```

Main iterative — visit twice

Lab postOrderIterativeS1, Solution02. This is the iterative postorder to use. It does not reverse a list.

On the way left, push the right child first as a reminder, then push yourself. You cannot record yourself yet.

When you pop yourself:

- reminder still on top → right side is not done. Put yourself back, go right.
- reminder gone → both sides are done. Record yourself.

```
stack = []
current = root
result = []
while current or stack:
    while current:
        if current.right:
            stack.append(current.right)
        stack.append(current)
        current = current.left
    current = stack.pop()
    if current.right and stack and stack[-1] is current.right:
        stack.pop()
        stack.append(current)
        current = current.right
    else:
        result.append(current.val)
        current = None
return result
```

current = None so the next loop does not walk left again.

Last — these reverse, and reverse is not guaranteed

They record too early (me, right, left), then flip to get left, right, me. That flip is the only thing that makes them postorder. Do not use them as the main way.

Two stacks

Assignment postOrderIterativeS2. First stack does not print. It saves every node on the second stack. Emptying the second stack is the flip. Push left then right, so the saved order is me, right, left.

```
s1.push(root)
while not is_empty(s1):
    current = pop(s1)
    push(s2, current)
    if current.left:
        push(s1, current.left)
    if current.right:
        push(s1, current.right)
while not is_empty(s2):
    print(pop(s2).data, end=" ")
```

Preorder is that first loop with two changes: it prints immediately, and it pushes right then left.

One stack, then reverse

Lab postOrderIterativeS1, Solution. A list is the only stack. reverse() is the flip.

```
result = []
if root is None:
    return result
stack = [root]
while stack:
    current = stack.pop()
    result.append(current.val)
    if current.left:
        stack.append(current.left)
    if current.right:
        stack.append(current.right)
result.reverse()
return result
```

recorded order = me, right, left after reverse = left, right, me The lab field is val.

Jobs that need postorder

These two need both children's answers before this node is done. Recursive only. They are not other ways to print postorder.

Max depth / height

1 + the taller child.

Tutorial Q4. Empty returns -1, so a single node is 0. The other file returns 0 for empty, so a single node is 1. Same 1 + the taller child.

```
if node is None:
    return -1
return 1 + max(height(node.left), height(node.right))
```

Has great-grandchild

that same height check. Print when a child is taller than 1. same walk. A child taller than 1 has a grandchild, so this node has a great-grandchild.

```
if node is None:
    return -1
left_h = has_great_grandchild(node.left)
right_h = has_great_grandchild(node.right)
if max(left_h, right_h) > 1:
    print(node.data, end=" ")
return 1 + max(left_h, right_h)
```

- leaf: both sides -1, height 0, no print
- parent of a leaf: child height 0, no print, this height is 1
- has a grandchild: child height 1, 1 is not > 1, no print
- has a great-grandchild: child height 2, print

When to use

Printing postorder: recursion, or visit twice. Visit twice is the iterative one. Two stacks and one-stack-then-reverse are last. They only become postorder after a flip, and reverse is not guaranteed. Height and great-grandchild need this order: the call returns the child's number, then this node uses it.

Order does not matter — this node is enough

These do not need inorder, preorder, or postorder. Any walk that sees every node once is fine. Print smaller than m and has grandchild are here, not under preorder. The order of the two sides does not matter.

Written as recursion that calls both children. A stack or the queue would do the same job. Those other versions are not written down.

Print smaller values than m	look at this node, then both children.
Has grandchild	does my child have a child? Then both sides.

Count nodes	add 1 for this node, plus both sides
Sum of non-leaf nodes	add me when I have a child
Smallest value	min of me and both sides
Mirror	swap this node's two children

Print smaller values than m

look at this node, then both children. The order of the two sides does not matter.

```
if node is None:
    return
if node.data < m:
    print(node.data, end=" ")
print_smaller(node.left, m)
print_smaller(node.right, m)
```

Has grandchild

Does my child have a child? look at this node. A child that itself has a child means this node has a grandchild. Then both sides, and collect the nodes that pass.

```
if root is None:
    return []
has = ((root.left and (root.left.left or root.left.right)) or
       (root.right and (root.right.left or root.right.right)))
if has:
    return [root.val] + has_grandchild(root.left) + has_grandchild(root.right)
return has_grandchild(root.left) + has_grandchild(root.right)
```

The stack version pops a node and does this same check, then pushes the children. The lab field there is val.

Count nodes

```
if node is None:
    return 0
return 1 + count(node.left) + count(node.right)
```

Sum of non-leaf nodes

a leaf adds nothing. Empty adds nothing.

```
if node is None:
    return 0
if node.left is None and node.right is None:
    return 0
return node.data + sum_non_leaf(node.left) + sum_non_leaf(node.right)
```

Smallest value

```
if node is None:
    return None
smallest = node.data
if node.left:
    smallest = min(smallest, smallest_value(node.left))
if node.right:
    smallest = min(smallest, smallest_value(node.right))
return smallest
```

This is any binary tree. On a BST the smallest is just keep going left.

Mirror

Swap first, then both sides. Each node is swapped once.

```
if node is None:
    return
node.left, node.right = node.right, node.left
mirror(node.left)
mirror(node.right)
```

When to use

Every node is visited, and the order is not what the question asks for. Add 1, add me if I have a child, take the min, or swap the two children. The empty child returns the empty answer: 0 for a count or a sum, None for a missing min.

Level order — one row, then the next

Level order	the queue. Take from the front. Children go at the back.
Kth-level descendants	not the queue. Recursive: count k down, then both children.

Level order

take from the front. Children go at the back. They are the next row. Tutorial Q2. Left is enqueued before right, so the row stays left to right.

```
if root is None:
    return
q.enqueue(root)
while not is_empty(q):
    current = dequeue(q)
    print(current.data, end=" ")
    if current.left:
        enqueue(q, current.left)
    if current.right:
        enqueue(q, current.right)
```

trace, the sample

- queue starts as 1
- take 1, enqueue 2 then 3. queue is 2 3
- take 2, enqueue 4 then 5. queue is 3 4 5
- take 3, take 4, take 5
- 1 2 3 4 5

stack: append and pop from the end = one side before the other queue: append and pop from the front = one row before the next

Kth-level descendants

the nodes k steps down. Count k down. At 0, this node is the row.

```
if node is None:
    return []
if k == 0:
    return [node.data]
return kth(node.left, k - 1) + kth(node.right, k - 1)
```

sample, k = 2: 4 and 5. k = 0 is the root itself.

When to use

The question is about a row, or about depth, and one side should not finish first. Whole tree row by row: a queue. Only the row k steps down: count k down, then both children.

BST — follow one child

Insert is the one with both ways. The others use one way. Smaller keys sit on the left. Bigger keys sit on the right. Search, insert, min, and max never visit both children.

Search	recursive. Compare, take one side.

Insert, recursive	walk until the child on that side is empty, then attach. An equal key is left alone.
Insert, iterative	same walk. Keep the parent. The empty spot has no node to hang the new one on.
Find min	iterative. Keep going left.
Find max	iterative. Keep going right.
Find LCA	recursive. Both smaller, go left. Both larger, go right. Else this node.
Remove	recursive down one side. Two children: find min of the right subtree, copy it, then remove that min.

Search

```
if node is None:
    return False
if node.data == value:
    return True
if value < node.data:
    return search(node.left, value)
return search(node.right, value)
```

Insert, recursive

```
if node is None:
    return Node(value)
if value < node.data:
    node.left = insert(node.left, value)
elif value > node.data:
    node.right = insert(node.right, value)
return node
```

The lab insert also writes it as: if the child is already None, attach here, else recurse. Same walk. The return has to be kept when the tree was empty.

Insert, iterative

insertBSTNodeVer2 is this write. The other iterative write attaches inside the loop as soon as the child is None.

```
if root is None:
    return Node(value)
current = root
parent = None
while current:
    parent = current
    if value < current.data:
```

```
            current = current.left
        elif value > current.data:
            current = current.right
        else:
            return root
    if value < parent.data:
        parent.left = Node(value)
    else:
        parent.right = Node(value)
    return root
```

Find min

```
if node is None:
    return None
while node.left:
    node = node.left
return node
```

Find max

```
if node is None:
    return None
while node.right:
    node = node.right
return node
```

Find LCA

```
if root is None:
    return -1
if p < root.data and q < root.data:
    return find_lca(root.left, p, q)
if p > root.data and q > root.data:
    return find_lca(root.right, p, q)
return root.data
```

trace

```
      6
     / \
    2   8
```

```
  / \
 1   4
```

LCA of 1 and 4: both smaller than 6, go left. 2 sits between them. answer 2. LCA of 1 and 8: 1 is smaller than 6 and 8 is larger, so 6.

The longer lab write sorts p and q first, then checks p < root < q. That is the same "else this node" case.

Remove

search first. Then one of three cases.

- no left child → the right child takes this spot
- no right child → the left child takes this spot
- two children → copy the min of the right subtree into me, then remove that min

The lab returns the node and a 0 or -1 as well. 0 means removed. -1 means missing. The tree change is still these three cases.

```
if node is None:
    return node, -1
if value < node.data:
    new_head, result = remove(node.left, value)
    node.left = new_head
    return node, result
if value > node.data:
    new_head, result = remove(node.right, value)
    node.right = new_head
    return node, result
if node.left is None:
    return node.right, 0
if node.right is None:
    return node.left, 0
succ = find_min(node.right)
node.data = succ.data
new_right, _ = remove(node.right, succ.data)
node.right = new_right
return node, 0
```

trace, remove 2 from the LCA tree

2 has two children. min of its right is 4. copy 4 into 2, then remove 4.

```
    6
  / \
 4   8
/
1
```

When to use

The tree is a BST, so one comparison picks the side. Search and insert: smaller left, bigger right, equal left alone. Min is the leftmost node. Max is the rightmost. LCA: while both keys are on the same side, go that side. The first split is the answer. Remove walks down one side. Two children: the min of the right subtree takes this node's value, then that min is removed.

On their own

Identical trees	recursive. Two trees, same step on both. Both sides must match.
Is BST (min/max range)	recursive. Both children, range passed down. The sorted check is the iterative inorder job, not this one.

Identical trees

both empty is a match. One empty is not. Values must match. Then both lefts and both rights.

```
if tree1 is None and tree2 is None:
    return True
if tree1 is None or tree2 is None:
    return False
if tree1.val != tree2.val:
    return False
return identical(tree1.left, tree2.left) and identical(tree1.right, tree2.right)
```

The lab field is val.

Is BST (min/max range)

every node must sit inside a range from its ancestors. Left child: same low, high becomes me. Right child: low becomes me, same high. Empty is a BST. The lab uses <= and >=, so an equal key is not a BST.

```
def is_bst(node, low=float("-inf"), high=float("inf")):
    if node is None:
        return True
    if node.data <= low or node.data >= high:
        return False
    return (is_bst(node.left, low, node.data)
            and is_bst(node.right, node.data, high))
```

isBSTWithHelper is this same check, with the first call passing -inf and inf.

When to use

Identical: two trees, and every step has to agree on both. Is BST by range: each node carries the allowed low and high down to its children. Is BST by sorted order: that is the inorder job. Each value bigger than the one

before.

Which technique when?

Sorted order, or left before me?	inorder. recursive, or one stack: push left, pop, step right
BST, kth smallest?	that same inorder. count the pops, stop at k
Each node becomes the sum of greater keys?	inorder with the sides swapped. running total
Print preorder?	this node, then both children. recursive, or one stack: pop, push right, push left
Print values smaller than m?	order does not matter. look at this node, then both children
Does this node have a grandchild?	order does not matter. does my child have a child, then both sides
Record me last?	postorder. visit twice is the iterative one. two stacks and one-stack reverse are last, because reverse is not guaranteed
Height?	needs postorder. 1 + the taller child. empty is -1 in the tutorial
Has a great-grandchild?	needs postorder. that height. print when a child is taller than 1
Count, sum of non-leaves, min, mirror?	order does not matter. see every node once
One row, then the next?	queue. take the front, children join the back
Only the nodes k steps down?	count k down, then both children. k == 0 is the row
BST search or insert?	compare, take one side. equal key left alone
BST min or max?	keep going left, or keep going right
BST lowest common ancestor?	both smaller go left, both larger go right, else this node
BST remove?	one side. two children: copy min of the right, remove that min
Two trees the same?	same step on both. both sides must match
BST by a range on each node?	low and high passed down. left tightens the high, right tightens the low